

Question-2

Part-A: Verifying a JWT token using a given secret

- JWT Token used:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMONTY3ODkwIiwibmFtZSI6Ikp5a2hpbCBLdWlhcilIsImhhdCI6MTUxNjIzOTYyMn0.d6kAltAy93sSoUQ6krNRrFcr8a7GYqQZak0LU_SbNDE
```

Secret Used to sign the above JWT Token: "nikhil"

- I generated the above JWT Token using the official JWT Website.
- Now, to verify a JWT token, we simply have to base64 encode a new signature, which is made using the payload and header, and then match it with the already existing signature in the JWT.
- First, I extracted the three fields of the JWT, which are splitted by the "."; these are the header, payload, and existing signature.
- Next, after getting the header, I decoded it using the `baseurl64_decode` function and then looked for the algorithm used in the hashing of this JWT.
- I have implemenetd checks for these algortimmsin my code:

```
AVAILABLE_ALGORITHMS = ["HS256", "HS384", "HS512"]
```

These are SHA256, SHA384, and SHA512, the numbers represent the bits generated after hashing a value.

- If the desired hashing algorithm exists in this list, then I proceed else raise a Runtime error.
- Next, I created a new signature using the parsed headers and payload, for this, I used the `hmac` library in Python.
- After generating the new signature, I encoded this, too, in base64 format and then matched this with the signature present in the JWT, if both these match, then the JWT is valid otherwise not valid.
- The following simple formula was used to generate the signature (taken from the official JWT site):-

```
HMACSHA256(  
    base64UrlEncode(header) + "." +  
    base64UrlEncode(payload),  
    secret)
```

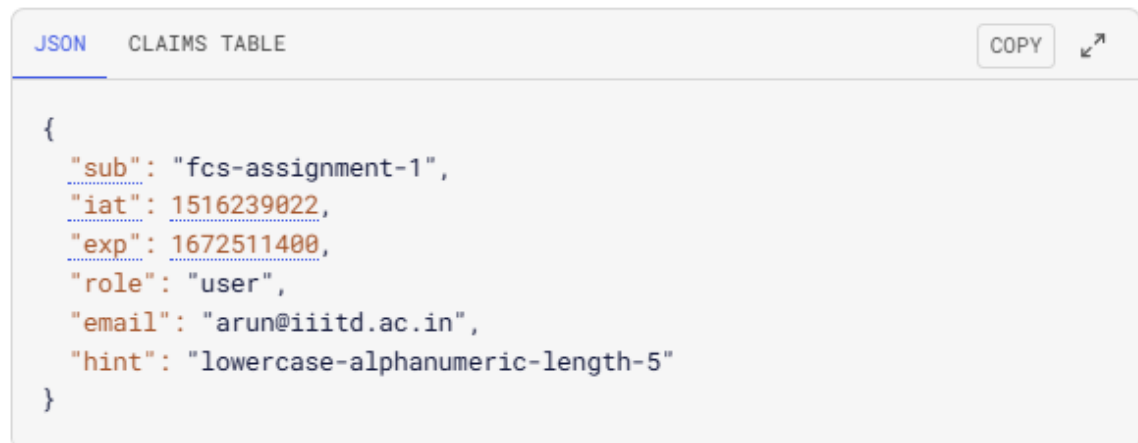
- Below is the payload of the JWT after successfully verifying the signature:-

```
{'sub': '1234567890', 'name': 'Nikhil Kumar', 'iat': 1516239022}
```

Part-B: Brute forcing the secret for the JWT

- I uploaded the given JWT on the official website, and it decoded the JWT and gave me a HINT in the payload:-

DECODED PAYLOAD



- Next, I brute-forced all the possible combinations of 5 lengths of alphanumeric characters using simple loops, and to verify the secret, I used the same function of part-a.
- After brute forcing, it gave me the secret as: "p1gzy".

```
JWT CRACKED!!!!
Key is: plgzy
Payload is: {'sub': 'fcs-assignment-1', 'iat': 1516239022, 'exp': 1672511400, 'role': 'user', 'email': 'a
run@iiitd.ac.in', 'hint': 'lowercase-alphanumeric-length-5'}
```

- After cracking the secret, I changed the payload by making the role admin and then created a new JWT Token in the same manner, mostly by utilizing the verifyJwt function's code. The formula remained the same.
- Below is the new JWT Produced:-

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiAiZmNzLWZfc2lnbmI1bnQtMSIsIChpYXQ
iOiAiXNTE2MjM5MDIyLCAiZXhwIjogMTY3MjUxMTQwMCwgInJvbGUiOiAiYWRTaW4iLCAiZW1h
aWwiOiAiYXJ1b2kiLCJ0ZC5hYy5pb25iLCJ0aW50IjogImxvd2VyY2FzZS1hbHB0YW51bWVyaW
MtbGVuZ3RoLTUifQ.LTc8VavAQR LcBRovG5kEJvVUOzMB3voQ17havY69ZE

- Below is the output from the official JWT site, where I verified this newly generated JWT against the secret key: p1gzy

ENCODED VALUE

JSON WEB TOKEN (JWT)

COPY CLEAR

Valid JWT

Signature Verified

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiAiZmWzLWFzc2lnbm1lbGQTMScisICjPvXQioiAXNTEZMjHSMdYlLCaIZXhwIjogMTYzMjUxMTQwMCwgZnVubGUoIAIyYWRtaW41LCaIZWlhawwioiAiyXJ1bkBpaWl0ZCshy5pbisICJoYW50IjogImxvd2VyY2FzZS1hbHB0YW51bnByamMtbgVuZ3RoLTUiFiQ.LTC8VayAQr_LcBRoyGSKEJVvuOzMB3voQ17havY69ZE

DECODED PAYLOAD

JSON CLAIMS TABLE

COPY ↗

{
 "alg": "HS256",
 "typ": "JWT"
}

DECODED PAYLOAD

JSON CLAIMS TABLE

COPY ↗

{
 "sub": "fcs-assignment-1",
 "iat": 1516239022,
 "exp": 1672511400,
 "role": "admin",
 "email": "arun@iiitd.ac.in",
 "hint": "lowercase-alphanumeric-length-5"
}

JWT SIGNATURE VERIFICATION (OPTIONAL)

Enter the secret used to sign the JWT below:

SECRET

COPY CLEAR

Valid secret

p1gzy

Part-C: Modifications to the JWT

- The first modification could be reducing the expiration time of the JWT; hence, JWT would be valid only for a short period of time now even if an attacker gets the secret key, but the JWT has expired, then the attacker just can't do anything to it.
- Next would be using multiple keys to validate a JWT; each JWT could have a keyID which would be unique, and at the same time, only one of those keys can be used to verify a JWT, like as a key rotation mechanism where keys are rotated timely.
- Another modification would be the use of asymmetric cryptography here; servers would sign a JWT using a private asymmetric key, and the client can only verify using a public key. This would reduce the dependency on only one key, and at the same time, it would ensure that the authentication system is manageable and robust.
- Another idea is the use of Multi-factor authentication in this architecture.

References

- JWT Official Website and Introduction to JWT Page:
<https://jwt.io/>
<https://jwt.io/introduction>